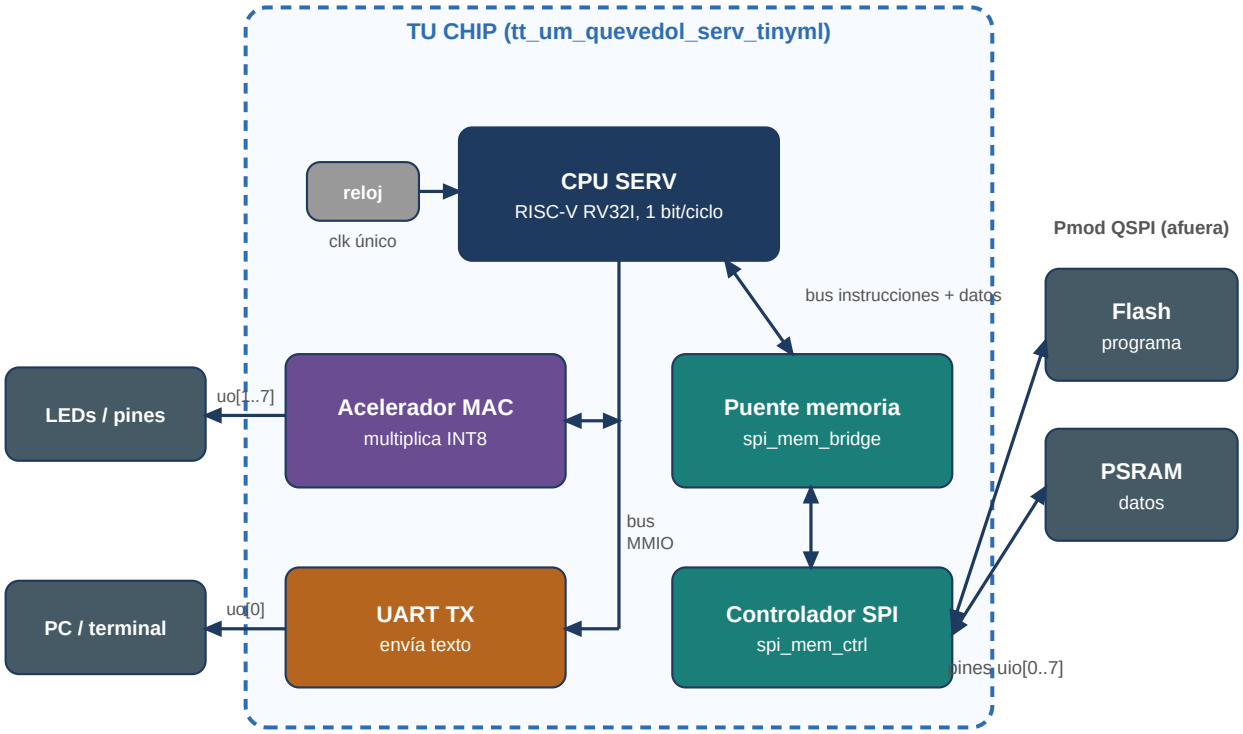
Tu chip explicado desde cero

Guía amigable del proyecto **SERV TinyML** para **Tiny Tapeout Sky130**



Qué hace el proyecto · qué herramientas se usan · cómo funcionan los pines y el tamaño (tiles) · qué hay que arreglar y cómo hacerlo

Repositorio: github.com/ric1806/tiny (copia de quevedol/tt-um-quevedol-serv-tinyml-sky130)
Versión actualizada el 28 de septiembre de 2026 (diseño corregido en la rama claude/hola-wl31sp)

Contenido

0. Antes de empezar: cómo leer esta guía 3

1. ¿Cómo se hace un chip? La idea general 4

2. ¿Qué es Tiny Tapeout? 5

3. ¿Qué hace este proyecto? 6

4. ¿Qué pasa cuando se enciende el chip? 8

5. Las herramientas, explicadas una por una 10

6. Los robots: GitHub Actions 12

7. Los pines, a fondo 13

8. El tamaño del diseño (tiles) 15

9. Lo que hay que arreglar, en orden 17

10. Cómo trabajar paso a paso 19

11. Glosario inglés -> español 20

0. Antes de empezar: cómo leer esta guía

Esta guía está pensada para alguien que estudia ingeniería electrónica, que conoce algunos conceptos (compuertas, flip-flops, reloj, microcontroladores) pero que **nunca ha trabajado con Tiny Tapeout** ni con diseño de chips, y que no domina el inglés. Por eso:

- Cada palabra en inglés que aparece en el proyecto se traduce y se explica la primera vez. Al final hay un **glosario** (sección 11) para consultarlas rápido.
- Los nombres de archivos, pines y comandos aparecen en `letra de código`, porque así los vas a ver en el repositorio.
- Los cuadros **azules** son analogías; los **amarillos** son advertencias; los **rojos** dicen qué tienes que hacer tú; los **verdes** son cosas que ya funcionan.

El orden recomendado de lectura es el del índice: primero entender qué es Tiny Tapeout (secciones 1 y 2), luego qué hace este chip (3 y 4), después las herramientas (5 y 6), y por último lo más importante para ti: **los pines y el tamaño** (7 y 8) y **la lista de cosas por arreglar** (9).

> **Ojo**
>
> Aunque todo el proyecto lo puedes leer en GitHub, la documentación original mezcla español e inglés y en algunos puntos **se contradice** (por ejemplo, unos archivos dicen que el chip mide 4x2 tiles y otro dice 3x2). Esta guía señala esas contradicciones: son justamente parte de lo que hay que arreglar.

> **Actualización del 28 de septiembre de 2026**
>
> Esta guía ya describe el diseño **corregido**. Lo que cambió respecto del proyecto original:
>
> - **Tamaño fijado en 3x2** (6 tiles); todos los documentos del repositorio lo dicen ya igual.
> - **Se quitó la pausa del reloj de SERV** (la compuerta de reloj). Era la causa de que el diseño físico fallara por tiempos de hold; la sección 4.2 lo explica.
> - El MAC tarda **9 ciclos** por multiplicación en vez de 8, para acortar su camino más lento.
> - Hay **pruebas nuevas** del chip completo, también en gate-level (documento "Pruebas del chip SERV TinyML").
> - Diseño físico en GitHub Actions: **aprobado** (commit c853560): gds, precheck y gl_test en verde; DRC, LVS y antenas limpios; hold cerrado en las 9 esquinas; utilización 82,9 %. Solo queda un aviso: un camino de setup se pasa 9,5 ps en la esquina más extrema (lenta, 100 °C, 1,6 V).

1. ¿Cómo se hace un chip? La idea general

Un chip digital (un microcontrolador, un procesador, etc.) no es más que millones de transistores conectados formando compuertas lógicas y flip-flops. Hoy en día nadie dibuja esos transistores a mano: se **describe el comportamiento** del circuito con un lenguaje de programación especial y unas herramientas automáticas lo convierten en los dibujos (planos) que la fábrica usa para fabricar el chip.

A ese lenguaje especial se le llama **HDL** (*Hardware Description Language*, lenguaje de descripción de hardware). En este proyecto se usa **Verilog**, que es el más común. Los archivos `.v` de la carpeta `src/` son Verilog.

> **Analogía para entenderlo**
>
> Piensa en construir una casa. El **Verilog** es como decirle al arquitecto "quiero 3 cuartos, 2 baños y una cocina conectada a la sala". Las **herramientas automáticas** son el arquitecto que dibuja los planos exactos (dónde va cada pared y cada tubo). El archivo **GDS** son esos planos finales. La **fábrica** (SkyWater) es la constructora que sigue los planos.

El camino completo, de la idea al chip, es este:

Paso	Qué pasa	En inglés lo verás como
1. Escribir RTL	Describes el circuito en Verilog (registros, sumadores, máquinas de estado…).	RTL = *Register Transfer Level*
2. Simular	Compruebas en la computadora que el circuito hace lo correcto, sin fabricar nada.	*simulation, testbench, test*
3. Sintetizar	La herramienta convierte tu Verilog en una lista de compuertas reales de la fábrica (AND, OR, flip-flops…).	*synthesis, netlist*
4. Colocar	Decide en qué lugar del chip va cada compuerta.	*placement*
5. Rutear	Dibuja los cables metálicos que conectan las compuertas.	*routing*
6. Verificar	Revisa que las señales lleguen a tiempo y que el dibujo cumpla las reglas de la fábrica.	*timing (STA), DRC, LVS*
7. GDS	Archivo final con los planos del chip.	*GDS, GDSII*
8. Fabricar	La fábrica produce el chip físico.	*tapeout, shuttle*

A los pasos 3 a 7 juntos se les llama **hardening** ("endurecer" el diseño: pasar de código a algo físico) o **diseño físico**. La palabra **tapeout** significa enviar los planos finales a fabricar.

2. ¿Qué es Tiny Tapeout?

Fabricar un chip propio normalmente cuesta decenas de miles de dólares. **Tiny Tapeout** es un proyecto educativo que abarata esto: junta los diseños de cientos de personas en **un solo chip grande**, y cada persona paga solo por el pedacito que ocupa.

> **Analogía para entenderlo**
>
> Es como un **edificio de apartamentos**. El chip completo es el edificio; cada diseño es un apartamento. Todos comparten la entrada, la luz y el agua (los pines externos, la alimentación), pero cada uno tiene su propio espacio. Un **tile** es como un "metro cuadrado" del edificio: si tu apartamento es más grande (más tiles), pagas más.

Conceptos clave de Tiny Tapeout

Palabra	Qué significa
Tile (baldosa)	La unidad de área. Un tile en Sky130 mide aprox. 160 × 110 micrómetros (el comentario de `info.yaml` dice 167×108 µm). Tu diseño ocupa un rectángulo de tiles, por ejemplo **3x2** = 3 de ancho por 2 de alto = 6 tiles.
Shuttle (lanzadera)	Cada "ronda" de fabricación. Cada shuttle tiene una fecha de cierre y un nombre/etiqueta, por ejemplo `ttsky26d`. Después del cierre ya no se aceptan cambios.
PDK	*Process Design Kit*: el "manual + biblioteca" de una fábrica. Incluye las compuertas disponibles, sus tamaños, retardos y las reglas de dibujo. Aquí se usa **Sky130** (sección 2.1).
Top module	El módulo Verilog "más externo" de tu diseño, el que se conecta a los pines. Debe llamarse `tt_um_...`. Aquí es `tt_um_quevedol_serv_tinyml` en `src/project.v`.
info.yaml	Archivo de configuración que le dice a Tiny Tapeout quién eres, cuántos tiles usas, cuál es el top module, qué archivos Verilog hay y qué hace cada pin. **Es el archivo más importante para la entrega.**
Demoboard	La placa que Tiny Tapeout entrega con el chip ya soldado. Trae un microcontrolador RP2040 que te permite seleccionar tu diseño, darle reloj, reset, y conectar cosas por los conectores **Pmod**.
Pmod	Un conector estándar de 12 pines para plaquitas de expansión. Aquí se usa el **QSPI Pmod** de Tiny Tapeout, que trae una memoria Flash y dos PSRAM.
GitHub Actions	Robots en la nube de GitHub que, cada vez que subes cambios, simulan y construyen tu chip automáticamente (sección 6).

2.1 ¿Qué es Sky130?

Sky130 (o **SKY130A**) es el proceso de fabricación de la empresa **SkyWater**, de **130 nanómetros**. "130 nm" es aproximadamente el tamaño del transistor más pequeño. Es una tecnología antigua (de principios de los 2000), pero tiene una gran ventaja: su PDK es **abierto y gratuito**, por eso Tiny Tapeout lo usa.

La biblioteca de compuertas que se usa se llama `sky130_fd_sc_hd`. Descifrado: **fd** = *foundry* (fábrica), **sc** = *standard cells* (celdas estándar, o sea compuertas prefabricadas), **hd** = *high density* (alta densidad, compuertas pequeñas). Cuando veas nombres como `sky130_fd_sc_hd__dlclkp_4`, es una compuerta concreta de esa biblioteca.

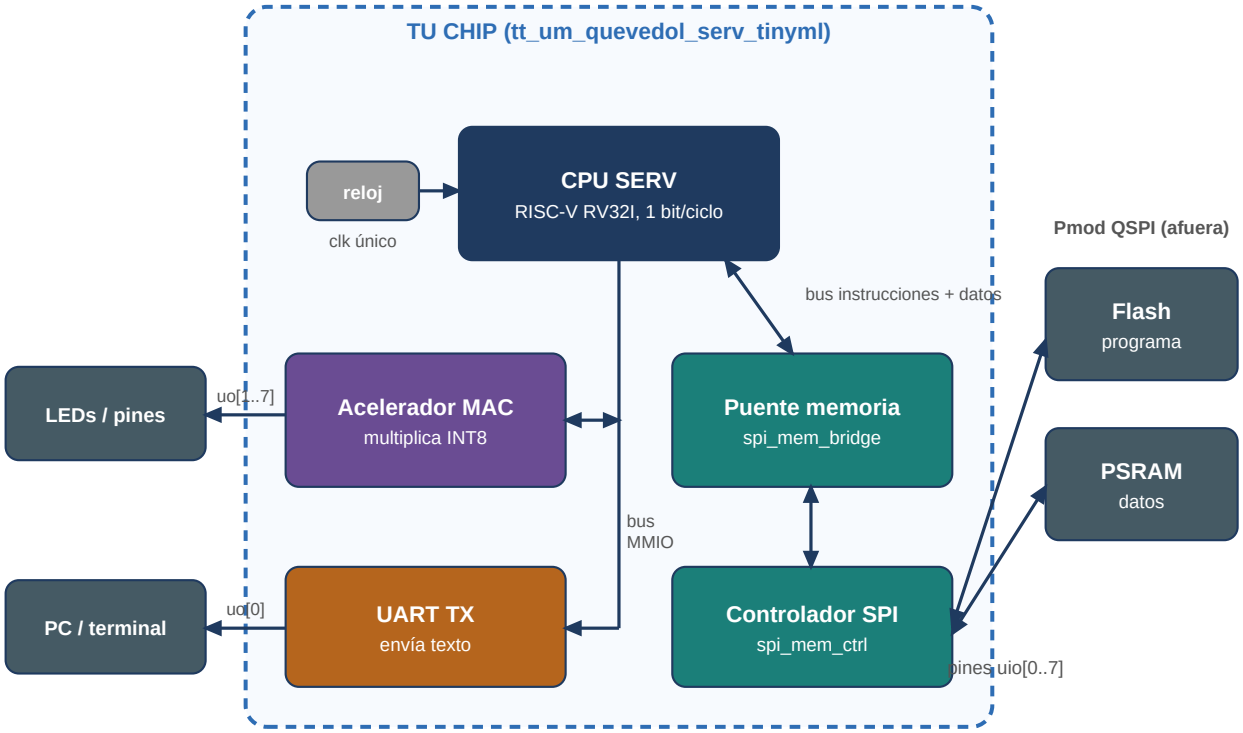
> **Ojo**
>
> Este proyecto antes estaba hecho para otra fábrica (**IHP**, de Alemania) y se migró a Sky130. Por eso existe el archivo `SKY130_MIGRATION.md`. Si algún día ves referencias a IHP o a `sg13g2`, son restos de la versión anterior.

3. ¿Qué hace este proyecto?

En una frase: **es un microcontrolador muy pequeño, con un ayudante para hacer cálculos de redes neuronales, que lee su programa de una memoria externa.**

El nombre del proyecto es **SERV TinyML SoC**:

- **SERV**: el procesador que usa. Es un procesador **RISC-V** famoso por ser el más pequeño del mundo.
- **TinyML**: *Tiny Machine Learning*, "aprendizaje automático diminuto": correr redes neuronales pequeñas en dispositivos muy pequeños.
- **SoC**: *System on Chip*, "sistema en un chip": procesador + periféricos (memoria, UART, acelerador) todos juntos.



3.1 Las piezas, una por una

a) El procesador SERV (RISC-V RV32I)

RISC-V es un conjunto de instrucciones abierto (como el ARM de los celulares, pero libre). **RV32I** significa: 32 bits, solo instrucciones enteras básicas (sumar, restar, comparar, saltar, leer y escribir memoria). No tiene multiplicación por hardware.

Lo especial de SERV es que es **bit-serial**: en vez de sumar los 32 bits de un número de golpe, los procesa **de 1 bit por ciclo de reloj**. Es unas 32 veces más lento, pero muchísimo más pequeño. Justo lo que se necesita cuando pagas por cada tile.

> **Analogía para entenderlo**
>
> Un procesador normal es como una autopista de 32 carriles: pasan 32 autos (bits) a la vez. SERV es una calle de **un solo carril**: pasan de uno en uno. Tarda más, pero la calle ocupa muchísimo menos terreno.

El código de SERV está en `src/cpu/serv/` y es una copia exacta de la versión oficial 1.4.0 (**no se debe modificar**). La configuración propia está en `src/cpu/serv_cpu.v`.

b) El acelerador MAC INT8

MAC = *Multiply-Accumulate* (multiplicar y acumular): hace la operación `acumulador = acumulador + a × b`. Es exactamente la operación que más se repite en una red neuronal (cada neurona suma entradas × pesos). **INT8** significa que *a* y *b* son enteros con signo de 8 bits (de -128 a 127); el acumulador es de 32 bits.

Además hace lo que necesita una neurona al final: sumar el **bias** (sesgo), aplicar **ReLU** (si el resultado es negativo, poner 0), hacer un **shift** (dividir entre 2^n) y **saturar** a 8 bits (si pasa de 127 se queda en 127). Para ahorrar área, cada multiplicación tarda **8 ciclos** (suma y desplaza, como la multiplicación a mano). Archivos: `src/accel/mac_int8.v` y `src/accel/mac_mmio.v`.

c) MMIO: cómo habla el procesador con el acelerador

MMIO = *Memory-Mapped I/O*, entrada/salida mapeada en memoria. El procesador no tiene instrucciones especiales para el acelerador: simplemente **escribe y lee en ciertas direcciones de memoria** que en realidad son registros del acelerador o de la UART. Es lo mismo que haces en un microcontrolador cuando escribes en un registro para prender un LED.

Dirección	Qué hay ahí
`0x0000_0000 - 0x00FF_FFFF`	Memoria **Flash** externa: el programa (16 MB)
`0x1000_0000 - 0x107F_FFFF`	**PSRAM A** externa: datos y variables (8 MB)
`0x2000_0000 - 0x2000_003F`	Registros del **acelerador MAC**
`0x2000_0100 / 0x2000_0104`	**UART**: dato a enviar / estado
`0x2000_0200`	Registro de estado: el programa escribe `0x534D4C31` ("SML1" en ASCII) cuando todo salió bien

d) La memoria externa (Flash y PSRAM por SPI)

El chip es tan pequeño que **no tiene memoria para el programa**. Todo vive afuera, en el **QSPI Pmod**:

- **Flash** (W25Q128JV, 16 MB): memoria no volátil. Guarda el programa (firmware) y, en el futuro, los pesos de la red neuronal. Es como el disco duro.
- **PSRAM** (APS6404L, 8 MB): memoria RAM. Guarda variables y resultados intermedios. Es como la memoria RAM de una computadora.

Se comunican por **SPI** (*Serial Peripheral Interface*), un protocolo serie con 4 cables: **SCK** (reloj), **MOSI** (datos del chip hacia la memoria), **MISO** (datos de la memoria hacia el chip) y **CS** (*chip select*, selecciona

con cuál memoria se habla; activo en 0). **QSPI** (*Quad SPI*) es la versión rápida con 4 cables de datos; el proyecto **todavía usa SPI simple** de 1 bit y deja reservados los otros dos (SD2, SD3).

Como leer una palabra de 32 bits por SPI tarda muchos ciclos, el módulo `spi_mem_bridge` le dice al procesador "espérate" (`cpu_wait`) y le retiene la respuesta (el *ack*): SERV simplemente espera hasta que llega el dato. Una versión anterior además le apagaba el reloj mientras tanto; en la sección 4.2 se explica por qué se quitó.

e) UART TX

UART es el puerto serie clásico (como el `Serial.print` de Arduino). **TX** = transmitir. El chip manda texto por el pin `uo[0]` a 115200 baudios (50 MHz / 434 ciclos por bit). Cuando el programa termina bien envía `SML1`.

f) El firmware y el modelo

El **firmware** (`firmware/`) es el programa en C que corre el procesador. **Bare-metal** significa "sobre el metal": sin sistema operativo. El modelo de red neuronal actual es de prueba (*smoke test*): una capa de **4 entradas -> 3 salidas** cuyo resultado esperado es `[32, 0, 19]`. El objetivo futuro es reconocer dígitos escritos a mano (**MNIST**, imágenes de 16×16) con una red `256 -> 16 -> 10`. En `model/` hay un programa en Python que calcula el resultado exacto para compararlo con el chip.

4. ¿Qué pasa cuando se enciende el chip?

Esta es la secuencia completa, paso a paso, que ya funciona en simulación:

1. La placa pone `rst_n = 0` (reset activo) al menos 10 ciclos de reloj. Mientras tanto, el chip mantiene los dos **CS** en 1 (ninguna memoria seleccionada) para no mandarles basura.
2. `rst_n` pasa a 1. El procesador SERV arranca en la dirección `0x0000_0000`, que corresponde a la Flash.
3. Para cada instrucción, el puente de memoria baja `CS Flash`, manda el comando SPI `0x03` (leer) + 24 bits de dirección, y recibe 32 bits. Mientras tanto SERV espera; su reloj sigue funcionando.
4. El código de arranque (`firmware/start.S`) copia las variables iniciales de la Flash a la PSRAM (comando `0x02` = escribir) y pone en cero la zona `.bss`.
5. Se ejecuta `main()`: prueba sumas, restas, XOR, desplazamientos, y luego usa el MAC para calcular la red 4 -> 3.
6. Si todo coincide con lo esperado, envía `"SML1\n"` por la UART y escribe la firma `0x534D4C31`. Si algo falla escribe `0xBAD0_xxxx` indicando qué falló.

> **Buena noticia**
>
> Ejecuté todas las pruebas de simulación del repositorio (RTL) el 28/09/2026 y **todas pasaron**: 10 pruebas en 6 grupos, incluyendo 10 000 multiplicaciones aleatorias comparadas con Python y el arranque completo desde Flash (tarda 2,65 ms simulados). El "cerebro" del diseño funciona; los problemas están en la parte física, los pines y la documentación.

4.1 Resultado de las pruebas que ejecuté

Grupo (Makefile)	Qué prueba	Resultado
`Makefile`	El top module: pines de memoria en reset y con `ena=0`	1/1 PASS
`Makefile.mac`	Acelerador MAC por MMIO + 10 000 productos aleatorios	2/2 PASS
`Makefile.spi`	Lectura SPI de una palabra de Flash	1/1 PASS
`Makefile.mem`	Puente de memoria: Flash vs PSRAM, prioridad, escritura	3/3 PASS
`Makefile.serv`	SERV corre el firmware C desde memoria ideal	1/1 PASS
`Makefile.extmem`	Arranque desde Flash/PSRAM serie + espera de SERV	2/2 PASS
`model/test_int8_reference.py`	Referencia Python del MAC	OK

4.2 Una lección importante: por qué se quitó la pausa del reloj

El diseño original **apagaba el reloj de SERV** mientras llegaba el dato de la memoria (*clock gating*). Apagar un reloj con una simple compuerta AND es peligroso (puede cortar un pulso a la mitad, un *glitch*), así que usaba una **ICG** (*Integrated Clock Gate*): la celda `sky130_fd_sc_hd__dlclkp_4`, que lleva un **latch** por dentro y solo cambia el permiso cuando el reloj está en bajo.

El problema apareció en el diseño físico. Detrás de esa compuerta quedaban 1 191 flip-flops con su propio reloj (`cpu_clk`), que llegaba con retraso respecto del reloj del resto del chip (`clk`). Ese desfase provocaba violaciones de **hold** en la esquina rápida de fabricación; el flujo metió 725 celdas de retardo y aun así no pudo cerrarlo.

Al revisarlo se comprobó que SERV **ya sabe esperar**: pide el dato y no avanza hasta recibir el *ack*. Sin la compuerta, el arranque completo funciona igual y tarda exactamente lo mismo. Por eso se quitó: ahora **todo el chip usa un solo reloj**, y con eso desaparecen las violaciones de setup y el hold real queda en unos pocos picosegundos, fácil de reparar.

> **Buena noticia**
>
> Lección de diseño: no dividas el reloj si no hace falta. Cada reloj extra es un problema de tiempos más para el diseño físico. El archivo `src/tech/sky130_clock_gate.v` sigue en el repositorio como referencia, pero ya no forma parte del chip.

4.3 Mapa de carpetas

Carpeta / archivo	Qué contiene
`info.yaml`	Configuración para Tiny Tapeout: tiles, top module, archivos, pines.
`src/project.v`	El **top module**: conecta el SoC a los 24 pines. **Aquí se decide qué hace cada pin.**
`src/config.json`	Opciones para la herramienta de diseño físico (LibreLane): frecuencia, densidad, márgenes de hold…
`src/soc/`	Integración: CPU + MMIO + UART + puente de memoria.
`src/cpu/`	Procesador SERV (`serv/` = copia oficial, no tocar).
`src/accel/`	Acelerador MAC.

Carpeta / archivo	Qué contiene
`src/mem/`	Controlador SPI y puente de memoria.
`src/periph/uart_tx.v`	Transmisor UART.
`src/tech/`	Compuerta de reloj de la versión anterior (ya no se usa).
`test/`	Pruebas automáticas (cocotb + Icarus).
`firmware/`	Programa en C, arranque en ensamblador, linker script.
`model/`	Referencia Python y modelo de prueba.
`docs/info.md`	Texto de la **hoja de datos** (datasheet) pública del proyecto.
`.github/workflows/`	Los robots de GitHub Actions (sección 6).
`.devcontainer/`	Entorno Docker con todas las herramientas instaladas.
`PROJECT_GUIDE.md, TASKS.md, SKY130_MIGRATION.md, STUDENT_HANDOFF.md`	Documentación del plan y el estado (en español).

5. Las herramientas, explicadas una por una

Todas las herramientas son gratuitas y de código abierto. No tienes que instalarlas todas: GitHub Actions ya las usa en la nube, y el `.devcontainer` trae un entorno listo.

5.1 Para escribir y simular (lo que usas día a día)

Herramienta	Para qué sirve	Cómo se usa aquí
Git / GitHub	Control de versiones: guarda la historia de cambios. GitHub es la página web donde vive el repositorio.	`git add, git commit, git push`. Cada `push` dispara los robots.
Verilog	Lenguaje para describir hardware.	Todos los `.v` de `src/`.
Icarus Verilog (`iverilog`)	Simulador: ejecuta el Verilog en tu computadora como si fuera el circuito real.	Lo llaman los `Makefile` de `test/`.
cocotb	Permite escribir las pruebas en **Python** en vez de Verilog: Python mueve los pines y revisa las salidas.	Archivos `test/test_*.py`. Versión 2.0.1.
GTKWave / Surfer	Visores de formas de onda: ves cada señal en el tiempo, como un osciloscopio/analizador lógico.	`gtkwave tb.fst tb.gtkw` después de simular.
RISC-V GCC	Compilador de C para el procesador RISC-V.	`firmware/Makefile` genera el `.hex` del programa.
Python	Referencia matemática del MAC y del modelo; pruebas.	`model/int8_reference.py`
Make	Automatiza comandos largos.	`make -f Makefile.mac`
Docker / devcontainer	Una "computadora virtual" con todo instalado para que no tengas que configurar nada.	Abrir el repo en VS Code -> "Reopen in Container".

5.2 Para el diseño físico (lo hacen los robots)

Herramienta	Para qué sirve
LibreLane	El "director de orquesta": un flujo que llama a todas las herramientas de abajo en orden, desde el Verilog hasta el GDS. Se configura con `src/config.json`. (Antes se llamaba OpenLane.)
Yosys	Síntesis: convierte el Verilog en compuertas de Sky130. También sirve para contar cuántas celdas ocupa el diseño.
OpenROAD	Hace el floorplan, la colocación (*placement*), el árbol de reloj (*CTS*), el ruteo y la reparación de tiempos. Los errores `GPL-xxxx` y `DPL-xxxx` vienen de aquí.
OpenSTA	Análisis de tiempos estático (*Static Timing Analysis*): revisa que todas las señales lleguen a tiempo (setup y hold).
Magic y **KLayout**	Editores/visores de layout. Revisan reglas de dibujo (**DRC**) y generan el GDS. KLayout sirve para ver tu chip en 2D.
Netgen	Hace el **LVS**: comprueba que el dibujo final corresponde exactamente al circuito sintetizado.
Precheck de Tiny Tapeout	Revisión final que exige Tiny Tapeout: pines en su lugar, tamaño correcto, sin errores.

5.3 Palabras del diseño físico que vas a ver en los reportes

Término	Explicación sencilla
Utilización (*utilization*)	Porcentaje del área disponible ocupado por compuertas. Si pasa de ~70-80 %, ya no quedan espacios para los cables y la herramienta falla. Si pasa de 100 %, simplemente **no cabe**.
Densidad objetivo (`PL_TARGET_DENSITY_PCT`)	Qué tan apretadas intenta poner las compuertas la herramienta. Por defecto 70 %.
Congestión	Demasiados cables queriendo pasar por la misma zona, como un embotellamiento.
Setup	La señal debe llegar al flip-flop **antes** del siguiente flanco de reloj. Si el circuito es muy lento hay violación de setup. Solución típica: bajar la frecuencia (`CLOCK_PERIOD`).
Hold	La señal **no debe cambiar demasiado pronto** después del flanco de reloj. Se arregla metiendo retardos (buffers). Los últimos commits del proyecto ajustan justamente esto (`*_HOLD_SLACK_MARGIN`).
Slack	Holgura de tiempo. Positivo = bien; negativo = violación.
CTS	*Clock Tree Synthesis*: construir la red que reparte el reloj a todos los flip-flops al mismo tiempo.
DRC	*Design Rule Check*: que el dibujo cumpla las reglas de la fábrica (distancias mínimas, anchos, etc.).
LVS	*Layout vs Schematic*: que el dibujo y el circuito sean el mismo.
Antenas	Cables metálicos muy largos pueden acumular carga durante la fabricación y dañar transistores; la herramienta añade diodos de protección.
Gate-level simulation	Simular el circuito **ya convertido a compuertas reales** (el netlist), no el Verilog original. Detecta errores que el RTL esconde.
Netlist	Lista de todas las compuertas y cómo se conectan.

6. Los robots: GitHub Actions

En `.github/workflows/` hay 4 archivos. Cada uno es una lista de pasos que GitHub ejecuta en una computadora en la nube cada vez que haces `git push`. Los resultados se ven en la pestaña **Actions** del repositorio en GitHub (un check verde OK: o una X roja X).

Archivo	Qué hace	Cuándo corre
`test.yaml`	Instala Icarus y cocotb y corre **todas las simulaciones RTL** (las mismas que ejecuté yo).	Cada push
`gds.yaml`	**El más importante.** Tiene 4 trabajos: **gds** (corre LibreLane y produce el chip), **precheck** (revisión de Tiny Tapeout), **gl_test** (simulación gate-level con `test/test.py`) y **viewer** (publica un visor 3D del chip en GitHub Pages).	Cada push
`docs.yaml`	Genera la hoja de datos a partir de `docs/info.md` e `info.yaml`.	Cada push
`fpga.yaml`	Genera un archivo para probar el diseño en una FPGA.	Solo manual (está desactivado)

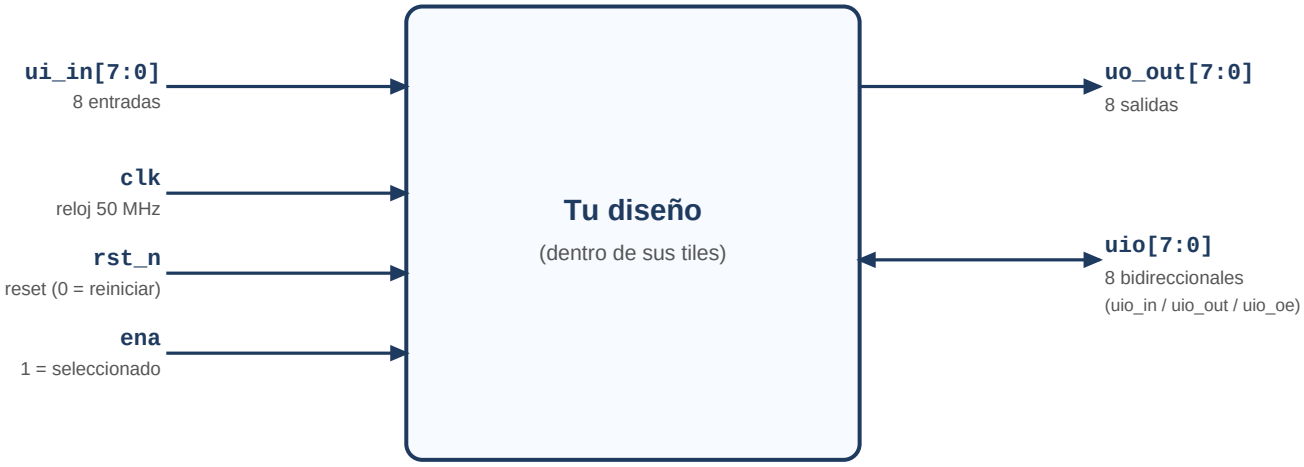
Cómo ver si tu chip "cabe" y cumple tiempos

1. Entra a tu repositorio en GitHub -> pestaña **Actions**.
2. Si dice que los workflows están desactivados, actívalos (en repos copiados GitHub los apaga por seguridad). Para el visor, habilita **GitHub Pages** en Settings -> Pages -> Source: **GitHub Actions**.
3. Abre la última ejecución de **gds**. Si falla, abre el paso rojo y lee el final del log: ahí aparece el código de error (por ejemplo `GPL-0301`).
4. Si pasa, en la página de resumen de la ejecución hay una tabla con la **utilización**, el número de celdas, y los resultados de timing y DRC. También puedes descargar el **artifact** (paquete de resultados) con el GDS y los reportes.

> **Ojo**
>
> Los robots usan etiquetas del shuttle: `@ttsky26d` en gds y docs, pero `@ttsky26c` en fpga. La documentación dice que hay que **cambiarlas por las del shuttle al que realmente se va a enviar**. En tu repositorio `ric1806/tiny` los robots funcionan y se pueden revisar desde la pestaña **Actions**. El trabajo **gds** ahora imprime al final de su log un resumen de tiempos (paso "Timing summary").

7. Los pines, a fondo

Esta es la sección clave para lo que te pidieron arreglar. En Tiny Tapeout **todos los diseños tienen exactamente los mismos pines**, sin importar su tamaño. No puedes agregar ni quitar pines; solo decides **qué hace cada uno**.



Total: 8 + 8 + 8 = 24 pines de usuario + clk, rst_n, ena

7.1 Los tres grupos de pines

Grupo	Dirección	En Verilog	Explicación
ui (*user input*)	Solo entrada	`ui_in[7:0]`	8 pines que llegan a tu diseño (botones, interruptores, RX…).
uo (*user output*)	Solo salida	`uo_out[7:0]`	8 pines que tu diseño maneja (LEDs, TX…). Siempre están como salida.

uio (*user in/out*)	Bidireccional	`uio_in[7:0]`
		`uio_out[7:0]`
		`uio_oe[7:0]`
			8 pines que pueden ser entrada o salida, **cada uno por separado**. Por eso tienen 3 señales (ver 7.2).

clk	Entrada	`clk`	Reloj. Aquí 50 MHz (`clock_hz` en info.yaml).
rst_n	Entrada	`rst_n`	Reset. La **_n** significa **activo en bajo**: con 0 se reinicia, con 1 funciona.
ena	Entrada	`ena`	Vale 1 cuando tu diseño está seleccionado y encendido. Este proyecto lo usa como reset extra: `soc_rst_n = rst_n && ena`.

7.2 Cómo funcionan los pines bidireccionales (uio)

Cada pin `uio[i]` tiene tres señales dentro de tu diseño:

- `uio_oe[i]` (*output enable*, habilitar salida): **1 = el pin es salida**, 0 = el pin es entrada (queda en alta impedancia, "desconectado").
- `uio_out[i]`: el valor que sacas por el pin **si** es salida. Si es entrada, se ignora.

- `uio_in[i]`: el valor que se lee del pin (sirve cuando es entrada).

> **Analogía para entenderlo**
>
> Cada pin uio es como una **puerta con un letrero**: `uio_oe` dice si la puerta es de "salida" o de "entrada". Si es de salida, por ella sale lo que pongas en `uio_out`. Si es de entrada, lo que llega se lee en `uio_in`. Si dos lados empujan a la vez (tu chip saca un valor y la memoria también) hay un **corto**: por eso es crítico que cada pin tenga la dirección correcta.

En este proyecto, `src/project.v` define:

```
assign uio_oe  = 8'b1100_1011;   // = 0xCB
assign uio_out = {1'b1, psram_a_cs_n_safe, 1'b0, 1'b0,
                  sck_safe, 1'b0, mosi_safe, flash_cs_n_safe};
```

Leyendo el número binario de **derecha a izquierda** (bit 0 es el de la derecha):

Pin	uio_oe	Dirección	Función actual	Señal QSPI Pmod estándar
`uio[0]`	1	Salida	CS de la Flash (activo en 0)	CS0 (Flash)
`uio[1]`	1	Salida	MOSI (datos hacia la memoria)	SD0 / MOSI
`uio[2]`	0	Entrada	MISO (datos desde la memoria)	SD1 / MISO
`uio[3]`	1	Salida	SCK (reloj SPI, 12,5 MHz)	SCK
`uio[4]`	0	Entrada	Reservado (SD2), sin usar	SD2
`uio[5]`	0	Entrada	Reservado (SD3), sin usar	SD3
`uio[6]`	1	Salida	CS de la PSRAM A (activo en 0)	CS1 (RAM A)
`uio[7]`	1	Salida	CS de PSRAM B, fijo en 1 (no seleccionada)	CS2 (RAM B)

> **Buena noticia**
>
> La asignación de los pines uio **coincide con el orden estándar del QSPI Pmod de Tiny Tapeout**, y las direcciones (`uio_oe`) son coherentes: los CS, SCK y MOSI son salidas; MISO es entrada; SD2/SD3 quedan como entrada para no chocar con la memoria. Además, durante el reset los CS se fuerzan a 1 (ninguna memoria seleccionada). La prueba `test/test.py` comprueba `uio_oe = 11001011` y los CS en 1.

7.3 Los pines de salida (uo) y de entrada (ui)

```
assign uo_out = {cpu_active, mac_overflow, mac_done, mac_result[3:0], uart_tx};
```

Pin	Función actual (project.v e info.yaml)	Lo que decía el plan original (PROJECT_GUIDE.md)
`uo[0]`	UART TX	UART TX
`uo[4:1]`	4 bits **bajos** del resultado del MAC	Clase predicha (dígito 0-9)
`uo[5]`	MAC terminó (`done`)	Inferencia terminada
`uo[6]`	Desbordamiento del MAC (`overflow`)	Error
`uo[7]`	CPU activa	CPU activa / debug

Pin	Función actual (project.v e info.yaml)	Lo que decía el plan original (PROJECT_GUIDE.md)
`ui[0]`	**Sin usar**	UART RX
`ui[1]`	**Sin usar**	Inicio manual de inferencia
`ui[2]`	**Sin usar**	Modo de prueba
`ui[3..7]`	Sin usar (vacío en info.yaml)	Sin usar

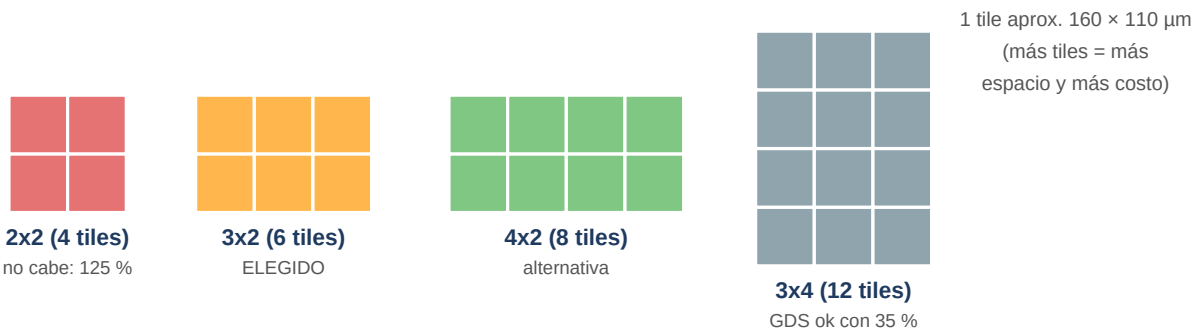
> **Ojo**
>
> Fíjate en `uo[4:1]`: muestra solo 4 de los 8 bits del resultado del MAC, y además **cambia en cada operación** y solo un instante. Por ejemplo, el resultado final del modelo es 19 = `0001_0011` en binario, y en los pines verías solo `0011` = 3. Y `uo[5]` (`mac_done`) es un bit **pegajoso**: sube con la primera multiplicación y se queda en 1 hasta el comando *clear*, así que no indica "inferencia terminada" (comprobado en simulación; ver el documento de pruebas). Son salidas útiles para depurar, pero **dicen poco en el chip real**.

7.4 Límites eléctricos que no debes olvidar

- Los pines trabajan a **3,3 V**. La Flash y la PSRAM del Pmod deben ser de 3,3 V.
- El reloj SPI es `SCK = 50 MHz / (2 × SPI_CLK_DIV) = 12,5 MHz`. La guía menciona "QSPI hasta 25 MHz" como objetivo, pero el diseño actual usa 12,5 MHz.
- Los pines de Tiny Tapeout pasan por multiplexores internos del chip grande, así que no pueden manejar señales muy rápidas; 12,5 MHz es una velocidad razonable, pero hay que confirmarla con la documentación del shuttle y el devkit.

8. El tamaño del diseño (tiles)

"Dimensionar" el diseño en Tiny Tapeout significa **elegir cuántos tiles ocupa**. Se declara en `info.yaml` con `tiles: "AxB"`. Si eliges pocos tiles, el diseño no cabe y la herramienta falla; si eliges muchos, funciona pero pagas más y ocupas espacio del chip compartido.



8.1 La historia de los intentos (según los commits)

Intento	Resultado	Explicación
2x2	NO: Falló `GPL-0301`	Las celdas suman 88 282 µm2 pero el área útil es 72 565 µm2: **125 %** de utilización. Físicamente no cabe.
3x2 (1ª vez)	NO: Falló `DPL-0036`	Se estimaba 81 % de utilización. Pasó la colocación global pero falló la colocación detallada (no encontró sitio exacto para todas las celdas), incluso subiendo la densidad a 80 y 90.
MAC serial	Cambio de diseño	Para ahorrar área, el multiplicador pasó de paralelo a uno que tarda 8 ciclos. Siguió fallando en 3x2.
3x3	No permitido	No es un tamaño válido en la plantilla Sky130.
3x4	OK: Generó GDS	35 % de utilización. Cabe con mucho espacio, pero son 12 tiles (caro).
4x2	Descartado	8 tiles. Se consideró como alternativa, pero se decidió quedarse en 3x2 (menos tiles).
3x2 con compuerta	NO: Hold	Coloca y rutea, con DRC, LVS y antenas limpios, pero falla por hold en la esquina rápida por el desfase entre relojes.
3x2 final	OK: Aprobado	Sin compuerta de reloj, un solo reloj, MAC de 9 ciclos. Ver la actualización de la sección 0.

> **Ojo**
>
> Tiny Tapeout solo acepta ciertos tamaños. Normalmente son 1x1, 1x2, 2x2, 3x2, 4x2, 6x2 y 8x2, pero **confírmalo en la página del shuttle** que vayan a usar. El 3x4 sirvió como experimento para medir el área, pero puede que no sea un tamaño aceptado para la entrega.

8.2 Cómo decidir el tamaño correcto

1. Ejecuta el workflow **gds** con el tamaño a probar y mira que pasen **todos** los pasos: colocación, CTS, ruteo, timing (setup y hold sin violaciones), DRC, LVS, antenas y precheck.
2. Mira la **utilización**. Lo sano es quedar por debajo de ~70 %, como dice la propia guía del proyecto. Con 3x2 se estaba en ~81 %: está al límite, y aunque pase, queda **sin espacio** para las funciones que faltan (QSPI de 4 líneas, UART RX, salidas de clase…).
3. Si 3x2 pasa, pregúntate: ¿cabrá lo que falta por agregar? Si la respuesta es no, lo prudente es **4x2**.
4. Cuando decidas, deja el **mismo número en todos los archivos** (sección 9, punto 1).

Decisión tomada: el proyecto se queda en **3x2**.

Si el diseño no cabe, la guía del proyecto propone este orden para recortar: quitar PSRAM B, quitar UART RX, quitar debug y contadores, usar multiplicador iterativo (ya hecho) y bajar la frecuencia a 32 MHz.

9. Lo que hay que arreglar, en orden

Esta lista sale de revisar todo el código, los documentos y el historial de cambios. Está ordenada por prioridad. Los puntos 1 y 2 son los que tienen que ver con el **dimensionamiento** y los **pines**.

1. El tamaño no coincidía entre archivos (RESUELTO)

Archivo	Qué dice sobre el tamaño
`info.yaml`	`tiles: "3x2"` (esto es lo que realmente se fabrica)
`README.md`	"targets a provisional **4x2** Tiny Tapeout block"
`PROJECT_GUIDE.md`	"Área reservada: **4x2 (8 tiles)**"
`TASKS.md`	"Obtener margen de área en **4x2**"
`SKY130_MIGRATION.md`	"Harden **4x2**", "el candidato actual pasa a 4x2"

> **Buena noticia**
>
> Resuelto: el tamaño quedó en **3x2** y los cinco archivos de la tabla ya lo dicen igual.

2. El pinout del plan no coincide con el pinout real

Hay dos "verdades" diferentes sobre los pines:

- **Lo que está implementado** (`src/project.v`, `info.yaml`, `docs/info.md`): `ui` sin usar; `uo[4:1]` = bits bajos del MAC; `uo[5]` = MAC done; `uo[6]` = overflow.
- **Lo que dice el plan** (`PROJECT_GUIDE.md`, tabla "Pinout objetivo"): `ui[0]` = UART RX, `ui[1]` = inicio, `ui[2]` = modo prueba, `uo[4:1]` = clase predicha, `uo[5]` = inferencia terminada, `uo[6]` = error.

Además, la Fase 3 del plan tiene sin marcar la tarea "Añadir salidas de clase, done y error". O sea, el pinout final **todavía no está implementado**.

> **Qué tienes que hacer**
>
> Decidir (con el supervisor, porque cambia el pinout) cuál de los dos será el definitivo. Recomendación: implementar el del plan, porque en el chip real sirve mucho más ver la **clase predicha estable** y un **done que signifique inferencia terminada** que ver resultados intermedios del MAC. Se podría hacer agregando un registro MMIO (por ejemplo en `0x2000_0204`) donde el firmware escriba la clase y los bits done/error, y sacar ese registro por `uo[6:1]`. Después, actualizar juntos: `src/project.v`, `info.yaml` (sección pinout), `docs/info.md`, `PROJECT_GUIDE.md` y la prueba `test/test.py`. **Cuidado:** cada cosa que añadas ocupa área, y con 3x2 ya estás al límite.

3. Detalles del info.yaml

- En `pinout`, `ui[0..2]` dicen "Unused" pero `ui[3..7]` están vacíos. La plantilla pide dejar los no usados en blanco: usa el mismo criterio para todos.
- `author: "quevedol"` y `discord: ""`: revisa si hay que poner el nombre público del autor (es una "decisión pendiente" del plan).

- Si cambias los archivos Verilog, recuerda que la lista `source_files` de info.yaml y `PROJECT_SOURCES` de `test/Makefile` deben coincidir.

4. Etiquetas del shuttle en los workflows

> **Qué tienes que hacer**
>
> `gds.yaml` y `docs.yaml` usan `@ttsky26d`; `fpga.yaml` usa `@ttsky26c`. Cuando el supervisor elija el shuttle, cambiar todas a la misma etiqueta oficial de ese shuttle (tarea "Seleccionar el shuttle Sky130" de `TASKS.md`).

5. Tiempos (timing) y clock gating (CORREGIDO)

El diseño físico en 3x2 fallaba por **hold** en la esquina rápida y avisaba de **setup** en la lenta. La causa era la compuerta de reloj de SERV (sección 4.2). Se quitó, el MAC acumula en un ciclo aparte, el margen de hold tras el árbol de reloj quedó en 0,02 ns (76 celdas; con más margen no caben en 3x2) y se activó una verificación extra tras el ruteo global.

> **Buena noticia**
>
> Estado: **aprobado** (commit c853560): gds, precheck y gl_test en verde; DRC, LVS y antenas limpios; hold cerrado en las 9 esquinas; utilización 82,9 %. Solo queda un aviso: un camino de setup se pasa 9,5 ps en la esquina más extrema (lenta, 100 °C, 1,6 V).

6. La prueba gate-level era muy básica (AMPLIADA)

`test/test.py` (la que corre en gate-level) solo comprueba `uio_oe` y que los dos CS estén en 1. No comprueba que el chip realmente arranque. Los comentarios del código dicen que hay valores `X` (desconocidos) en algunos pines al simular con compuertas.

> **Buena noticia**
>
> Hecho: `test/tb_board.v` y `test/test_board.py` conectan el modelo de Flash/PSRAM al chip completo, comprueban que por `uo[0]` salga `SML1` y también corren en gate-level (documento de pruebas).

7. Documentación desactualizada

- El calendario de `PROJECT_GUIDE.md` fijaba el congelamiento para el **19 de septiembre de 2026**, y ya pasó. Hay que actualizar las fechas según el shuttle real.
- "QSPI hasta 25 MHz" en la guía vs. 12,5 MHz reales.
- Varias tareas terminadas no están marcadas o están desactualizadas (por ejemplo, el registro de identificación `SML1` en `0x2000_0200` ya existe, pero la Fase 3 lo tiene sin marcar).

8. Hardware externo (antes de fabricar)

- Confirmar que tienen el **QSPI Pmod** y que se conecta a los pines uio de la demoboard.

- Documentar cómo se **programa la Flash** con el firmware antes de encender el chip (la hoja de datos dice que falta este procedimiento).

10. Cómo trabajar paso a paso

10.1 Preparar tu computadora

Opción fácil (recomendada): instala **VS Code** y **Docker Desktop**, abre la carpeta del repositorio y elige "**Reopen in Container**". Todo queda instalado.

Opción manual en Linux o WSL (Windows):

```
sudo apt-get install -y iverilog gtkwave
pip install -r test/requirements.txt
```

10.2 Correr las pruebas

```
cd test
make -B                       # prueba del top module
make -f Makefile.mac -B       # acelerador MAC (tarda ~15 s)
make -f Makefile.spi -B       # lector SPI
make -f Makefile.mem -B       # puente de memoria
make -f Makefile.serv -B      # procesador con memoria ideal
make -f Makefile.extmem -B    # arranque completo desde Flash/PSRAM
```

Al final de cada una verás una tabla con `PASS` o `FAIL`. Luego, para ver las señales:

```
gtkwave tb.fst tb.gtkw
```

10.3 Probar el diseño físico

1. Haz tu cambio (por ejemplo en `info.yaml` o `src/config.json`).
2. Guarda y sube: `git add -A && git commit -m "Mi cambio" && git push`.
3. Ve a la pestaña **Actions** de GitHub y espera (el gds puede tardar 15-40 minutos).
4. Revisa el resumen: utilización, timing, DRC, precheck y gl_test.

10.4 Buenas prácticas del proyecto

- **No edites** `src/cpu/serv/`: es la copia oficial de SERV.
- Haz **un cambio pequeño a la vez** y describe qué hiciste (`CONTRIBUTING.md`).
- Si cambias RTL o firmware, agrega o actualiza una prueba.
- Pide revisión antes de cambiar: el top module, los pines, `info.yaml`, los tiles, el shuttle, la configuración de SERV o la arquitectura de reloj (`STUDENT_HANDOFF.md`).
- Actualiza `TASKS.md` y `PROJECT_GUIDE.md` cuando termines algo.

10.5 Recursos (en inglés, pero con buenos diagramas)

- tinytapeout.com: FAQ, guías y la página de cada shuttle (tamaños permitidos, fechas).
- tinytapeout.com/digital_design: lecciones de diseño digital.
- tinytapeout.com/guides/local-hardening: correr LibreLane en tu computadora.

- Discord de Tiny Tapeout: la comunidad responde muy rápido.
- Consejo: usa el traductor del navegador; los términos técnicos se quedan en inglés, y para eso tienes el glosario de la siguiente página.

11. Glosario inglés -> español

Término	Significado
ASIC	Chip hecho a la medida para una tarea (lo contrario de una FPGA reprogramable).
Bare-metal	Programa que corre sin sistema operativo.
Bias	Sesgo: número que se suma al resultado de una neurona.
Bit-serial	Procesa un bit por ciclo de reloj.
Bring-up	Primera puesta en marcha del chip real en el laboratorio.
Chip select (CS)	Señal que elige con qué dispositivo SPI se habla. Activa en 0.
Clock gating	Apagar el reloj de una parte del circuito.
Commit	Un cambio guardado en Git, con mensaje.
Core	Área interior del diseño donde van las compuertas.
CTS	Clock Tree Synthesis: red de distribución del reloj.
Datasheet	Hoja de datos.
DRC	Design Rule Check: revisar reglas de dibujo de la fábrica.
Firmware	Programa que corre dentro del chip.
Flash	Memoria no volátil (no se borra al apagar).
Floorplan	Plano base: tamaño del chip y ubicación de pines y alimentación.
GDS	Archivo final con el dibujo del chip.
Gate-level (GL)	Simulación con las compuertas reales, después de la síntesis.
Hardening	Convertir el Verilog en un diseño físico (GDS).
Hold / Setup	Reglas de tiempo de los flip-flops (ver sección 5.3).
ICG	Integrated Clock Gate: celda especial para apagar relojes sin glitches.
INT8 / INT32	Enteros con signo de 8 / 32 bits.
Latch	Elemento de memoria que es transparente mientras su habilitación está activa.
Linker script	Archivo que dice en qué dirección de memoria va cada parte del programa.
LVS	Layout vs Schematic: comparar dibujo contra circuito.
MAC	Multiply-Accumulate: multiplicar y acumular.
MISO / MOSI	Master In Slave Out / Master Out Slave In: líneas de datos del SPI.
MMIO	Periféricos manejados escribiendo/leyendo direcciones de memoria.
Netlist	Lista de compuertas y sus conexiones.
Output enable (oe)	Señal que convierte un pin bidireccional en salida.
PDK	Kit de diseño de la fábrica: celdas, reglas, modelos.

Término	Significado
Placement	Colocar las compuertas en el chip.
Pmod	Conector estándar para plaquitas de expansión.
PSRAM	RAM externa de bajo costo que se accede por SPI.
Pull request (PR)	Propuesta de cambios en GitHub para que alguien los revise.
Push	Subir tus commits a GitHub.
ReLU	Función de activación: si x<0 da 0; si no, da x.
Routing	Dibujar los cables entre compuertas.
RTL	Register Transfer Level: descripción del circuito en Verilog.
Shuttle	Una ronda de fabricación de Tiny Tapeout.
Slack	Holgura de tiempo; negativa = error.
Smoke test	Prueba mínima para ver si "sale humo" (si funciona lo básico).
SoC	System on Chip: sistema completo en un chip.
Standard cell	Compuerta prefabricada de la biblioteca del PDK.
Tapeout	Enviar el diseño a fabricar.
Testbench	Circuito/programa que prueba tu diseño en simulación.
Tile	Unidad de área en Tiny Tapeout.
Tri-state / alta impedancia	Pin "desconectado", ni 0 ni 1.
UART RX / TX	Puerto serie: recibir / transmitir.
Utilization	Porcentaje de área ocupada por compuertas.
Waveform	Forma de onda: gráfica de señales en el tiempo.
Workflow	Lista de pasos automáticos de GitHub Actions.

Guía basada en el proyecto original (commit 4ae8464) y actualizada con las correcciones del 28 de septiembre de 2026. Las pruebas RTL se ejecutaron con Icarus Verilog y cocotb 2.0.1; el diseño físico, con LibreLane en GitHub Actions.